

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

APPENDIX C

RELATIONS WITH MODULES | SERVERPOD

 Copy as Markdown

Relations with modules

Serverpod [modules](#) usually come with predefined tables and data structures. Sometimes it can be useful to extend them with your data structures by creating a relation to the module tables. Relations to modules come with a restriction: you do not own the module's table definition, so you cannot change its structure.

Since you do not directly control the models inside the modules, it is recommended to create a so-called "bridge" table linking the module's model to your own. This is set up the same way as a normal [one-to-one](#) relation. For example, to attach your own profile data to the signed-in user from the [authentication module](#):

```
class: UserProfile
table: user_profile
fields:
  authUser: module:serverpod_auth_core:AuthUser?, relation(onDelete=Cascade)
  age: int
indexes:
  user_profile_auth_user_idx:
    fields: authUserId
    unique: true
```

Or by referencing the table name if you only want to access the id:

```
class: UserProfile
table: user_profile
fields:
  authUserId: UuidValue, relation(parent=serverpod_auth_core_user)
  age: int
indexes:
  user_profile_auth_user_idx:
    fields: authUserId
    unique: true
```



Ask AI

Note that the foreign key's type must match the module table's id type, and `AuthUser` uses a `UuidValue` id. If you assign the module a nickname in your `generator.yaml`, use that nickname in the reference instead. See [modules](#).

It is now possible to make any other relation to your model as described in [one-to-one](#), [one-to-many](#), [many-to-many](#), and [self-relations](#).

Advanced example

A one-to-many relation combined with the bridge table could look like this:

```
class: UserProfile
table: user_profile
fields:
  authUser: module:serverpod_auth_core:AuthUser?, relation(onDelete=Cascade)
  age: int
  company: Company?, relation(name=company_employee)
indexes:
  user_profile_auth_user_idx:
    fields: authUserId
    unique: true
  company_unique_idx:
    fields: companyId
    unique: true
```

```
class: Company
table: company
fields:
  name: String
  employees: List<UserProfile>?, relation(name=company_employee)
```

For the full walkthrough of relating your data to signed-in users, see [Working with users](#).